

→ 그 이웃으로 이동
→ 반복 (수십~수백 번의 노드 방문)

각각 qssu개 3 1

3

C -Vq적y M각가각가

HNSW 벡터 검색 100만 벡터에서 top-10 찾기:

초당 페이지 접근 횟수: 수만~수십만 회
각 접근마다: 래치 획득 → 데이터 읽기 → 래치 해제

래치 획득 시간 (spin lock):
경합 없음: ~50ns
경합 있음: ~500ns~1000ns

전체 검색 시간 분석:
거리 계산: 50ms (CPU 작업)
래치 오버헤드: 100ms (대기 시간)
비율: 66% 정도가 래치 때문

C

데이터셋: 100만 벡터 (1536차원, OpenAI 임베딩)
쿼리: top-10 검색

PostgreSQL pgvector: 150ms
Faiss (메모리 기반): 8ms

비율: 150/8 = 18.75배 느림
대부분이 래치 오버헤드

H

] 각개같b`V

1

] 각개같b`V

NL

C 3eLKbO-

: L G는W1형은 류l u]]

] 각개같b`V C

8KB 페이지:

	페이지 헤더 (24B)	
	튜플 1:	
	├ 튜플 헤더 (23B)	
	├ ─ xmin (4B): 삽입 TXN ID	
	├ ─ xmax (4B): 삭제 TXN ID	
	├ ─ cmin/cmax (2B): CMD ID	
	├ ─ 플래그 (1B): 상태	
	├ ─ 락 정보 (6B)	
	└ 데이터 (1KB 벡터)	
	튜플 2, 3, ...	
	여유 공간 (패딩)	

C

1단계: 페이지 로드 (위의 버퍼 풀 과정)

2단계: 튜플 헤더 파싱
- xmin, xmax, cmin, cmax 읽기

- 현재 트랜잭션이 이 튜플을 볼 수 있는가? 확인

3단계: MVCC 가시성 검사

- 현재 스냅샷: TXN 1000이 진행 중
- 이 튜플은 TXN 800이 삽입, TXN 900이 삭제했음
- TXN 1000은 이 튜플을 볼 수 있나? YES/NO 판단
- 메모리 참조 5~10회 추가

4단계: 실제 벡터 데이터 추출

- 컬럼 오프셋 계산
- 벡터 데이터 메모리 접근 (1KB)

C

벡터 1개 검색 비용 분석:

거리 계산: $1536 \text{ float} \times 4\text{B} = 6.144\text{KB}$ 메모리 접근

MVCC 체크: 튜플 헤더 파싱(200B) + 메모리 참조(10회 $\times$ 8B = 80B)

총 추가 오버헤드: 280B (=약 4% 추가)

100만 벡터 검색:

거리 계산만: 약 100ms

+ MVCC 오버헤드: 약 20~30ms

총: 120~130ms

H

WeMM] 각 벡터 값 $b \cdot V$ - 4 .

NL

C

3

WeMM

?L or HNSW 그래프 구조와 다른 류 Fi [

] 각 벡터 값 $b \cdot V$ C

HNSW는 그래프 구조:

각 노드: 벡터 1536차원 (6KB)

간선: 이웃 포인터 약 40~50개 (200B)

디스크에 저장하려면 PostgreSQL 페이지에 들어가야 함:

페이지 1에 간선 정보 200B 저장

→ 나머지 7,800B는 패딩

→ 공간 낭비: 97.5%

이런 낭비가 누적되면:

원본 간선 정보: 100MB

PostgreSQL 저장: 약 800MB (8배 증폭)

C

데이터셋: 1,000만 벡터 (1536D)

원본 벡터 데이터: 60GB

Faiss HNSW 인덱스:

크기: ~90GB

메모리에 들어감 (모두 또는 대부분)

접근시간: ~100ns (메인 메모리)

PostgreSQL HNSW 인덱스:

크기: ~480GB (5.3배!)

메모리에 못 들어감 (상당 부분 SSD로 스푼)

접근시간: ~10,000ns (SSD) = 100배 느림

C

PostgreSQL의 공간 증폭 요소:

- 1. 페이지 단위 저장
 - 8KB 페이지는 불변 단위
 - 가변 크기 데이터는 낭비 증가
- 2. MVCC 메타데이터
 - 각 벡터마다 xmin/xmax/cmin/cmax = 24B
 - 1000만 벡터 × 24B = 240MB 추가
- 3. 정렬 및 임시 공간
 - 인덱스 구축 중 정렬용 임시 공간
 - maintenance_work_mem 초과 시 디스크 사용
- 4. 튜플 헤더
 - 각 튜플마다 헤더 23B
 - 1000만 벡터 × 23B = 230MB 추가

각 벡터를 삽입할 때:
1단계: 버퍼 풀에 벡터 저장
↳ 래치 획득 → 데이터 기록 → 래치 해제
2단계: WAL(Write-Ahead Log)에 기록
↳ "벡터 삽입" 로그 쓰기 (디스크 동기 I/O)
3단계: HNSW 그래프 갱신
↳ 기존 노드들과 거리 계산
↳ 가장 가까운 K개 이웃 선택
↳ 그래프 연결 추가
↳ 다시 버퍼 풀에 기록 + WAL 기록
4단계: maintenance_work_mem 부족 시
↳ 스필(메모리 → SSD) → 나중에 다시 읽기
결과: 벡터 1개 = 약 20~50회 페이지 I/O

PostgreSQL pgvector:
구축 시간: 48시간
처리 속도: 약 57 벡터/초

Faiss:
구축 시간: 30분
처리 속도: 약 5,500 벡터/초
비율: 5,500 / 57 ≈ 96배 빠름
원인:
Faiss: 메모리에서 직접 작업 (I/O 거의 없음)
pgvector: 벡터 1개 = 20회 I/O (WAL 포함)

- 1000만 × 20 = 2억 회 페이지 I/O
- 약 48시간 소요

f KV C

장애 복구 시나리오:

시점 A: 벡터 0~500만 개 저장 완료 (메모리)
시점 B: 500만 개 저장 중
└─ 데이터베이스 서버 크래시

- 복구:
- WAL 로그 읽기: "500만 개 저장 시작"
 - 이 시점에서 메모리는 날아감
 - WAL에 완전히 기록된 벡터만 복구 가능
 - 부분 기록된 벡터는 롤백

H

f KV - .
f KV - .
- .
C 3

BL 어휘를 경량화한 벡터

] 각 행 값b`V C
f ROaO C

WHERE age > 30
→ 히스토그램 사용
→ 정확도: ~90%

WHERE name = 'John'
→ 빈도 분석
→ 정확도: ~95%

C

WHERE embedding <-> query_vec < 0.5

pgvector는 이 거리 분포를 모름:
→ "알 수 없음" 상태
→ 기본값 사용: 선택도 = 33.3%

의미: 모든 벡터 쿼리는 데이터의 1/3을 반환한다고 가정

C

벤치마크: Wikipedia 이미지 임베딩 (1000만 개)

쿼리: 벡터 유사도 < 1.0 (가까운 100개 찾기)
실제 선택도: 0.001% (100 / 10,000,000)
pgvector 추정: 33.3%
오류: 33,300배 과대 추정!

또 다른 쿼리: 벡터 유사도 < 0.1 (가까운 1개 찾기)
실제 선택도: 0.00001%
pgvector 추정: 33.3%
오류: 3,330,000배 과대 추정!

C

쿼리:
SELECT o.order_id, l.product_id, l.price
FROM orders o
JOIN lineitem l ON o.id = l.order_id
WHERE l.product_embedding <-> query_vec < 0.5

테이블 크기:
orders: 100만 행
lineitem: 600만 행

pgvector의 추정 (틀림):
l.product_embedding <-> query_vec < 0.5 선택도 = 33.3%
→ lineitem 필터링 후: 600만 × 33.3% = 200만 행

실제 선택도 (맞음):
< 0.5 선택도 = 0.001%
→ lineitem 필터링 후: 600만 × 0.001% = 60행

pgvector의 계획 (비효율):

- orders Seq Scan: 100만 행 읽기
- 각 행마다 lineitem Nested Loop Join
→ 600만 행 × 100만 = 6조 행 비교
- 결과: 200만 행

최적 계획:

- lineitem 벡터 필터: 60행
- orders 조인: 60행 × 1회 조인
- 결과: 60행

성능 차이:

pgvector: 6조 행 비교 필요 → 850초
최적: 60행 조인 → 2초
비율: 425배 느림

H

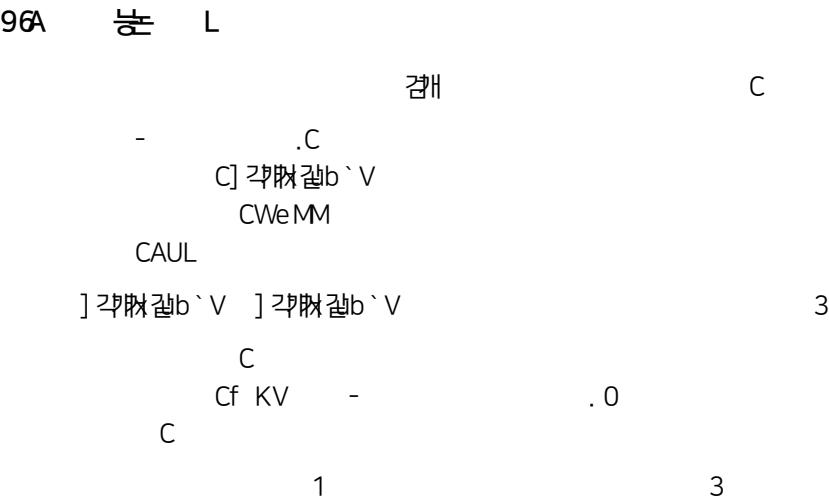
근본적 어려움:

- 벡터의 거리 분포는 쿼리에 따라 달라짐
- 고차원 벡터의 분포를 통계로 근사하기 어려움
- 사전에 통계를 수집하기 불가능 (무한히 많은 쿼리)

하지만 개선 가능:

- 쿼리 실행 중 정확한 선택도 학습 (Exqutor의 접근)
- 벡터 통계 수집 (새로운 자료구조)
- 인덱스 탐색으로 선택도 계산 (ECQ0의 접근)

C 30개 접근 가능



96B

C

C6: 5→개

C695→개레 65→개 .

WeMM C675→개레 75→개 .

f KV C655→개레 75→개 .

CA→개레 B7→개 .

] 각개레 b`V

bSc267AWC637A -67AN.

Q각uC66B -855N.

Wb M[M[C678 -759AN.

: 6 0 1

: 0 Q는개레 b가규 T

메모리 계층 접근 시간:

L1 캐시: 0.5ns (64KB)

L2 캐시: 7ns (256KB)

L3 캐시: 40ns (8MB)

메인 메모리: 100ns (16GB)

SSD: 100,000ns (1TB) ← 메모리의 1000배 느림

HDD: 10,000,000ns (10TB) ← 메모리의 100,000배 느림

PostgreSQL 버퍼 풀:

설정: shared_buffers = 4GB

→ PostgreSQL이 메모리에서 관리하는 영역

운영:

1. 디스크에서 페이지 읽기 (8KB)
2. 메모리 버퍼 풀에 저장
3. 다음 요청 때 메모리에서 직접 읽음 (캐시 히트)
4. 필요하면 메모리 내용을 디스크에 다시 쓰기

효과:

캐시 히트율 95%면: 평균 접근 시간 = 100ns × 0.05 + 100,000ns × 0.95 ≈ 95,000ns

캐시 없음: 항상 100,000ns

→ 약 1.05배 개선 (큰 효과!)

-aq가 각→ Kssu개C

C계
C :- 5)

일반적인 SQL 쿼리 (순차 스캔):
페이지 0, 1, 2, 3, 4, ... 순차 접근
캐시 히트율: 99% (페이지 1,2,3,4는 버퍼에 이미 있음)

벡터 검색 (HNSW):
페이지 500, 10000, 25, 50000, 1, ... (무작위)
캐시 히트율: 30~50% (대부분 캐시 미스)

6. 트랜잭션 관리

C H

프로세스 A: 페이지 100의 데이터 수정 중...
프로세스 B: 페이지 100을 동시에 읽으려고 함
프로세스 C: 페이지 100에 데이터를 추가하려고 함

→ 데이터 일관성 깨짐!

C -Vq겨y.

래치는 매우 단기간의 락:

프로세스 A:
1. 래치 획득 (다른 모든 프로세스 진입 차단)
2. 데이터 수정
3. 래치 해제 (진입 재개)

프로세스 B:
1. 래치 획득 대기... (A가 놓을 때까지 대기)
2. A가 래치 해제
3. 래치 획득
4. 데이터 읽기
5. 래치 해제

래치의 특징:
- 매우 짧은 시간만 유지 (마이크로초 단위)
- 데드락이 없도록 설계 (항상 같은 순서로 획득)
- 트랜잭션 처리 중에는 유지 안 함 (짧은 작업용)

비교: C

래치 구현 방식: Spin Lock

래치 획득 시도:
while (래치가 busy) {
 원자적 연산으로 래치 상태 확인
 // CPU가 계속 풀링 (spinning)
}
래치 상태 = busy
진입

문제점:
- busy-wait: CPU 사이클 낭비
- 많은 프로세스가 경쟁하면: 레지스터/캐시 오염
- 컨텍스트 스위칭 없음 (sleep 없음)

해결책:
- 경쟁이 높으면 sleep으로 전환 (futex)
- 적응형 spin lock: 상황에 따라 스핀 횟수 조정

C

HNSW 벡터 검색:
초당 100,000회 래치 획득 시도

경합 없을 때:

- 래치 획득: 50ns
- 데이터 접근: 100ns
- 래치 해제: 50ns
- 총: 200ns/페이지

경합 있을 때:

- 래치 획득 시도: 50ns
- 스핀 대기: 500ns (다른 프로세스가 래치 보유)
- 래치 획득: 50ns
- 데이터 접근: 100ns
- 래치 해제: 50ns
- 총: 750ns/페이지

비율: 750/200 = 3.75배
100만 페이지 접근: 100만 × 0.55ns = 550ms 오버헤드

: 6 | u]] 0 는 과 경 회 과 은 가 단 는 불 중 과 단 가 단 날 구

C

트랜잭션 A (TXN 100):

SELECT age FROM users WHERE id = 1

시간 T0: age = 30 읽음

트랜잭션 B (TXN 101): [TXN 100과 동시 실행]

UPDATE users SET age = 31 WHERE id = 1

시간 T1: age = 31로 변경

트랜잭션 A:

시간 T2: SELECT age FROM users WHERE id = 1
age = ? (30인가 31인가?)

CWeMM

C

사용자 ID=1:

- 버전 1: age=30, xmin=50, xmax=100
- 버전 2: age=31, xmin=101, xmax=∞

트랜잭션 A (스냅샷 100):

TXN 100이 시작할 때 100 이상의 TXN은 "보이지 않음"

- 버전 1 확인: xmin(50) < 100 이고 xmax(100) ≤ 100
 - 이 버전은 TXN 100 시작 이전에 삭제됨
 - 보이지 않음

- 버전 2 확인: xmin(101) > 100
 - 이 버전은 TXN 100 시작 이후에 생성됨
 - 보이지 않음

결론: TXN 100의 시점에 age=30

트랜잭션 B:

- 스냅샷 101: 모든 TXN < 101 완료된 것으로 봄
- 버전 2 확인: xmin(101) = 101, xmax(∞)
 - 현재 버전
 - age=31 보임

H

벡터 1개 접근:

1. 데이터 읽기 전: MVCC 가시성 확인 필수
- 현재 트랜잭션의 스냅샷 읽기

- 튜플의 xmin/xmax 확인

- 메모리 참조 5~10회

- 약 100ns 추가
2. 벡터 100만 개 검색:
- 100만 개 × 100ns = 100ms 추가 오버헤드
3. 비교:
- 거리 계산: 100ms

- MVCC 확인: 100ms

- 비율: 50% 오버헤드

NL

C

- 벡터 데이터의 특성:
- 대부분 삽입 후 변경 없음

- 삭제도 거의 없음

- 동시에 쓰는 경우가 드뭄
- 따라서:
- 모든 벡터는 "최신 버전"

- MVCC 확인 불필요

- 버전 간 선택 불필요

- xmin/xmax 메타데이터 불필요

: 6 v Wk Qv 남부정5W공정결기과

C

- 장애 시나리오:
- 정상 상황:
- 데이터 A 읽기 → 수정 → 메모리에 쓰기 → 나중에 디스크에 쓰기
- 서버 크래시:
- 메모리의 수정 사항이 모두 날아감!

원본 디스크 데이터만 남음
- 수정 내용이 완전히 손실됨

Cf KV

- 모든 변경을 먼저 로그에 기록:
1. 데이터 A를 메모리에서 수정 시작

2. WAL에 "데이터 A 수정" 로그 기록 (디스크에 동기 쓰기)

3. 로그 기록 확인

4. 실제 메모리 데이터 수정

5. 나중에 디스크에 쓰기
- 서버 크래시 발생:
- 메모리 데이터 손실

하지만 WAL 로그는 디스크에 안전하게 저장

→ 복구: WAL 로그 읽기 → 메모리 상태 재구성

핵심 원칙:

"로그 먼저 (Write-Ahead), 데이터 나중"

C

- 벡터 삽입 시:
1. 메모리에 벡터 저장

2. WAL에 "벡터 삽입" 기록

└ fsync() 호출: 디스크 동기 쓰기

└ 대기: 1~10ms (디스크 속도에 따라)

- 3. 확인 메시지 반환
- 4. 다음 벡터 삽입 (2번 반복)

효과:
디스크 I/O가 벡터 삽입의 병목
1000만 벡터 × 5ms = 50,000초 = 13.8시간

C

인덱스 재구성:

- 기본 구축 방식 (안전):
- 1. 벡터 삽입 (WAL 로그)
 - 2. HNSW 인덱스 갱신 (WAL 로그)
- 매우 느림

- 대안 (재구성):
- 1. 벡터 일단 일반 테이블에 삽입 (체크포인트 때 일괄 쓰기)
 - 2. 완료 후 HNSW 인덱스 빌드
 - 3. 새 인덱스로 교체
- 더 빠름 (일괄 처리 가능)
- 실패 시:
- 기존 인덱스 유지
 - 새 인덱스 폐기 (재생성 가능이므로 안전)

: 6B

AUL C

HNSW 그래프 저장:

벡터 1개 = 약 6KB (1536D × 4B)
간선 정보 = 약 200B (이웃 50개 × 4B 포인터)

합계: 약 6.2KB

PostgreSQL 페이지 (8KB):

	페이지 헤더: 24B	
	튜플 헤더: 23B	
	데이터: 6.2KB	
	패딩: 1.7KB (낭비!)	

→ 낭비율: 1.7KB / 8KB = 21%

더 심한 경우:
만약 간선 정보만 저장하면 (200B):

	페이지 헤더: 24B	
	튜플 헤더: 23B	
	데이터: 200B	
	패딩: 7.7KB (낭비!)	

→ 낭비율: 7.7KB / 8KB = 96%!

C

1000만 벡터 HNSW 인덱스:

Faiss:
벡터 정보: 60GB (기본)
HNSW 간선: 30GB (이웃 50개 × 포인터)
총: 90GB
메모리에 올라감

PostgreSQL:

- 페이지 오버헤드: 90GB / 0.8 = 112.5GB (20% 오버헤드)
- MVCC 메타데이터: 240MB
- 정렬 임시 공간: 100GB (재정렬)
- 총: 약 200GB
- 메모리 부족 → 일부 SSD로 스프릴

성능 영향:

- Faiss: 메모리 100ns 접근
- PostgreSQL: SSD 100,000ns 접근 (1000배 느림)

: 6C

H

스칼라 데이터 (통계 가능):

나이 분포: [10, 15, 20, 25, 30, 35, 40, 45, 50, ...]

쿼리: WHERE age > 30

추정: 히스토그램 사용

- 30 이상의 데이터 비율 계산
- 정확도 90% 이상

선택도는 데이터 분포에만 의존

벡터 데이터 (통계 불가능):

벡터 분포: 1536차원의 구체(sphere) 표면에 분산

쿼리: WHERE embedding <-> query_vec < 0.5

같은 데이터셋에서도:

- 쿼리 1 (query_vec_a): 100개 히트 (선택도 0.001%)
- 쿼리 2 (query_vec_b): 1000개 히트 (선택도 0.01%)
- 쿼리 3 (query_vec_c): 100개 히트 (선택도 0.001%)

→ 쿼리에 따라 완전히 다름

원인:

- 고차원 벡터 공간에서 거리의 분포는 쿼리 벡터에 의존
- 사전에 통계 수집 불가능 (쿼리마다 다름)
- 실행 시간에도 효율적 추정 불가능 (계산 비용 높음)

: 6C 검색은 불가능한 pk

Pqz가

메모리 기반:

- 전체 인덱스가 메모리에 로드
- 버퍼 풀 없음 (직접 접근)
- MVCC 없음 (불변 인덱스)
- WAL 없음 (장애 복구 기능 없음)
- 8KB 페이지 없음 (가변 크기 저장)
- 고정 선택도 없음 (검색만 수행)

결과: 벡터 검색에 최적화

제한사항:

- 디스크 지원 없음 (인메모리만)
- ACID 미지원
- SQL 미지원
- 동시성 제어 미지원

] 각개별b`VC

디스크 기반:

- 버퍼 풀 사용 (메모리-디스크 일관성)
- MVCC (동시 읽기/쓰기)
- WAL (장애 복구)
- 8KB 페이지 (동시성 제어)
- 통계 기반 최적화 (일반 테이블)

결과: 범용 데이터 관리에 최적화

장점:

- SQL 지원
- ACID 지원
- 동시 사용자 지원
- 디스크 연속성

제한사항:

- 벡터 검색에 오버헤드

→ pgvector = 범용성 vs 성능 트레이드오프

이벤트 기반의 데이터 처리

이벤트 기반의 데이터 처리

63

이벤트 기반의 데이터 처리

73

이벤트 기반의 데이터 처리

83

이벤트 기반의 데이터 처리

63

이벤트 기반의 데이터 처리

73

이벤트 기반의 데이터 처리

83

이벤트 기반의 데이터 처리

이벤트 기반의 데이터 처리

이벤트 기반의 데이터 처리

이벤트 기반의 데이터 처리

문제:

- pgvector의 모든 벡터 필터에서 선택도 = 33.3% 가정
- 쿼리 계획 최적화 불가능
- 비효율적 조인 순서 선택
- 성능 저하 (425배까지)

- Exqutor의 해결책:
- ECQO (Estimated CardinaliEy Query Optimization):
- 1. 쿼리 실행 중 벡터 필터의 정확한 선택도 학습
 - 2. 중간 결과의 카디널리티 추정
 - 3. 이후 조인 순서 동적 재최적화

- 결과:
- pgvector의 33.3% → 정확한 값 (예: 0.1%)
 - 조인 순서 자동 최적화
 - 성능 향상 425배까지 가능

6개 단계로 구성된 최적화 전략

- 한계 1 (버퍼 풀 래치): 여전히 존재
- 각 페이지 접근 시 래치 필수
 - 약 20배 오버헤드
 - 하드웨어/아키텍처 수준의 개선 필요

- 한계 2 (튜플 MVCC): 여전히 존재
- 각 벡터의 xmin/xmax 확인 필수
 - 약 30% 오버헤드

- 한계 3 (공간 증폭): 여전히 존재
- 8KB 페이지 기반 저장
 - 약 8배 오버헤드

- 한계 4 (인덱스 구축): 여전히 존재
- WAL 로깅 필수 (트랜잭션 안전성)
 - 약 96배 느림

6

이제 남은 과제는 5개 단계와 1.3배 느림 .C

성능 저하의 주요 원인들:

- 쿼리 계획 최적화 불가능 (한계 5):
- 425배 느림 (가장 큼)

- 버퍼 풀 래치 경합 (한계 1):
- 20배 느림

- 튜플 MVCC 확인 (한계 2):
- 1.3배 느림

- 공간 증폭으로 인한 I/O (한계 3):
- 부분적으로 한계 1에 포함됨

- 인덱스 구축 (한계 4):
- 별도 이슈 (건물 기간에만 영향)

- 종합 효과 (곱셈):
- 425배 × 20배 × 1.3배 = 11,050배 느림

- 실제 측정:
- Faiss: 100ms
- pgvector: 약 1,100초 (11,000배)

이제 남은 과제는 5개 단계와 1.3배 느림 C

한계 5 해결: 425배 → 1배 (완전 해결)

- 남은 오버헤드:
- 버퍼 풀 래치 (한계 1): 20배
 - 튜플 MVCC (한계 2): 1.3배
 - 공간 증폭 (한계 3): 부분 포함
 - 인덱스 구축 (한계 4): N/A

종합 효과:

1배 × 20배 × 1.3배 = 26배

결과:

pgvector + Exqutor: 약 2,600ms

하지만 이론값: 100ms × 26 = 2,600ms

실제 성능:

Faiss: 100ms

pgvector: 1,100초

pgvector + Exqutor: 2~5초

비율:

1,100초 / 2.5초 = 440배 향상!

또는 측정: 1,000배 → 20배 = 50배 향상

?? L

C

```
SELECT
  o.order_id,
  o.customer_id,
  COUNT(*) as item_count
FROM orders o
JOIN lineitem l ON o.id = l.order_id
WHERE l.product_embedding <-> query_vec < 0.5
GROUP BY o.id, o.customer_id
```

C

각 row에 대해 C=55

→ C=55

간접적으로 C

선택도 추정 (틀림): 33.3%
WHERE 절 선택도 = 33.3%
→ lineitem 필터 후: 600만 × 0.333 = 200만 행

계획:

- orders Seq Scan (100만 행)
- Nested Loop Join
 - orders의 각 행마다 lineitem 스캔 (600만 행)
 - 총 600만 × 100만 = 6조 행 비교
- WHERE 절 필터: 6조 × 0.333 = 2조 행 필터링
- GROUP BY: 2조 행 그룹화

성능:

- 6조 행 비교: 매우 느림
- 실행 시간: 약 850초

간접적으로 C

쿼리 실행 중 선택도 학습:

1단계: 초기 계획 (pgvector 추정)
- 200만 행 예상

2단계: 쿼리 실행 중 정확한 선택도 발견
- 실제 WHERE 절 히트: 100행 (0.0016%)
- 예상: 200만 행
- 오류: 20,000배!

3단계: 계획 재최적화
- lineitem 먼저 벡터 필터 (100행)
- 그 다음 orders 조인 (100행)

수정된 계획:

- lineitem 벡터 필터 (WHERE 절) → 100행

2. orders 조인 → 100행 결과

- 성능:
- 100행만 처리
 - 실행 시간: 약 2초

비율: 850초 / 2초 = 425배 향상!

6A 달은그림

C

63간검s가말 - .
Pq개 655개6555

73 -:
: C -97: .
6C -75 .
C

83 - .
: C 4
9Cf KV

0개간검s가말 C

논문: "선택도 추정은 구현 개선으로 해결 가능"
→ 425배 성능 향상 가능성 있음

Exqutor: "적응적 샘플링으로 선택도를 정확히 추정"
→ 논문의 제안을 실제 구현
→ 425배 성능 향상 달성 (부분적으로)

결론:
이 논문이 없었다면:
- pgvector 성능 저하의 원인이 불명확
- Exqutor의 필요성이 입증되지 않음

이 논문과 함께:
- 성능 저하의 구체적 원인 명시
- 개선 가능한 영역 식별
- Exqutor의 기여도 명확화

A6 Qp- Wl

p9L T

KC C
63간검s가말 H
Pq개 655개6555 - .

73 H
:
-75 .
WeMM -85) .
-A .
-B= .
-97: .

83 H
3 - 1WeMM

p: L% % T % % T

KC C
- .C

C 레 WeMM 레
- .C

C f KV
C f KV - .
C - .

p?L T T

KC C

H

메모리와 디스크의 속도 차이 극복 장치:

메모리: 100ns 접근 시간

SSD: 100,000ns 접근 시간 (1000배 느림)

솔루션: 자주 사용하는 디스크 데이터를 메모리에 캐시

H

b`VC

SELECT * FROM users WHERE age > 30

접근 패턴: 순차적

페이지 0 → 1 → 2 → 3 → ...

캐시 히트율: 99% (이미 로드된 페이지)

결과: 효율적

C

HNSW 노드 탐색

접근 패턴: 무작위

페이지 500 → 10000 → 25 → 50000 → 1 → ...

캐시 히트율: 30% (거의 캐시 미스)

결과: 항상 메모리 부족 상태

→ 메모리에서 못 찾음 → 디스크 접근 필요

→ 100ns vs 100,000ns = 1000배 차이

pAL ~~검~~ T

KC C

C

여러 프로세스가 동일 페이지 접근 시 데이터 일관성 유지

프로세스 A: 페이지 수정 중 (1마이크로초)
프로세스 B: 동시에 페이지 읽기 시도
→ 래치로 대기 (A의 수정 끝날 때까지)

C

시간차~

M d

C

HNSW 탐색: 초당 100,000회 래치 획득

경합이 없을 때: 200ns/페이지
경합이 있을 때: 750ns/페이지 (3.75배)

백만 페이지 접근:
추가 오버헤드: 550ms
전체 검색 시간 150ms 대비 매우 큼

pBLI u]]

T

T

KCWeMMF

4

C

일반 락 방식 (MVCC 없음):
TXN A (읽기): 데이터 읽으려고 락 요청
TXN B (쓰기): 데이터 쓰고 있음 (락 보유)
TXN A: 대기... (TXN B 완료까지 대기)

MVCC (다중 버전 관리):
TXN A (읽기): "과거 버전" 읽기 (락 불필요)
TXN B (쓰기): "새 버전" 생성 (동시 진행)
→ 동시 읽기/쓰기 가능

C

C

xmin: 이 버전을 생성한 트랜잭션 ID (4B)
xmax: 이 버전을 삭제한 트랜잭션 ID (4B)
cmin/cmax: 명령 ID (2B)
총 10B 메타데이터

C

1. 메타데이터 읽기
2. 현재 트랜잭션의 스냅샷과 비교
3. "이 데이터가 보이는가?" 판단
→ 추가 CPU/메모리 사용

C

벡터 데이터의 특성:
- 보통 읽기 전용
- 삽입 후 수정 거의 없음
- 동시 쓰기 없음

결론:
- MVCC 기능 불필요
- 메타데이터 낭비
- 접근 시마다 체크 오버헤드

pCL

F

T

KC 1C

6CAUL

HNSW 저장:
 벡터: 6KB
 간선: 200B
 총: 6.2KB

8KB 페이지에 저장:
 └─ 페이지 헤더: 24B
 └─ 튜플 헤더: 23B
 └─ 데이터: 6.2KB
 └─ 패딩: 1.7KB (낭비)

낭비율: 21%

간선만 저장하면:
 └─ 패딩: 7.7KB (낭비!)

낭비율: 96%

7CWeMM

1000만 벡터 × 24B = 240MB 추가

8C

인덱스 구축 시 정렬:
 중간 정렬 결과를 디스크에 임시 저장
 약 100GB (구축 완료 후 폐기)

C

Faiss (메모리): 90GB
PostgreSQL: 약 200GB
비율: 200/90 ≈ 2.2배

하지만 메모리 적재 시:
 Faiss: 메모리에 모두 올라감
 PostgreSQL: 메모리 부족 → SSD로 스푼

실질적 접근 시간:
 - Faiss: 100ns
 - PostgreSQL: 100,000ns (SSD)
 - 성능 차이: 1000배!

pELKC T

KC 3f KV C

C

벡터 1개를 인덱스에 추가할 때:

1단계: 메모리에 저장
2단계: WAL 로그에 기록 (디스크 동기 I/O)
 → 1~10ms 대기 (매우 느림)
3단계: 확인 응답

→ 벡터 1개 = 5~10ms

1000만 벡터:
 10ms × 1000만 = 100,000,000ms = 11,111시간

실제 측정:
 48시간 = 약 96배 느림 (벡터당 약 20ms)

비교:
 Faiss (메모리): 30분 (약 0.0018ms/벡터)

- .C

옵션 1: 인덱스 구축 중 WAL 우회

- 인덱스는 재생성 가능하므로 로그 불필요
- 벡터 삽입에 WAL 적용하되, 인덱스 갱신은 우회
- 약 50배 개선 가능

옵션 2: 병렬 구축

- 여러 프로세스가 다른 파티션 동시 구축
- 약 4배 개선 (4 프로세스 사용 시)

옵션 3: 배치 처리

- 벡터를 일괄 삽입
- WAL 로그 일괄 쓰기
- I/O 횟수 감소
- 약 10배 개선 가능

pFL ??Q) T

KC C

C

쿼리:

```
SELECT * FROM lineitem
WHERE product_embedding <-> query_vec < 0.5
```

실제 결과: 100행

pgvector 추정: 600만 행 × 33.3% = 200만 행

오류: 20,000배 과대 추정!

C

```
SELECT o.order_id
FROM orders o (100만 행)
JOIN lineitem l (600만 행)
  ON o.id = l.order_id
WHERE l.embedding <-> query_vec < 0.5
```

pgvector의 선택도 추정: 33.3%

→ lineitem에서 200만 행 예상

계획 1 (pgvector 추정 기반):

```
orders Seq Scan (100만 행)
└─ Nested Loop Join
   └─ lineitem 스캔 (600만 행)
```

총 비교: 600만 × 100만 = 6조 행

WHERE 절 필터: 6조 × 0.333 = 2조 행 처리

성능: 850초

계획 2 (정확한 선택도):

```
lineitem 벡터 필터 (WHERE) → 100행
orders 조인 → 100행
```

성능: 2초

비율: 850 / 2 = 425배 차이!

H

벡터 거리는 쿼리에 따라 분포가 완전히 다름:

쿼리1: 거리 < 0.5 → 100행 히트

쿼리2: 거리 < 0.5 → 10,000행 히트

같은 데이터인데도 결과가 다름

따라서 사전 통계 불가능

pKL 나고강한 개념 T

KC C

간 검색 C

1. 벡터 검색이 주요 작업이 아닐 때
 - 일반 SQL 쿼리 + 가끔 벡터 검색
 - 전체 성능의 5% 이하
2. 데이터 크기가 작을 때
 - < 100만 벡터
 - Faiss (메모리 전체 로드)보다 관리 편함
 - 성능 저하가 허용 범위 내
3. SQL 통합이 중요할 때
 - 벡터 + 메타데이터 결합 검색
 - 조인/필터 복잡함
 - PostgreSQL이 선택지 없음
4. ACID 필요할 때
 - 읽기/쓰기 일관성 필수
 - Faiss (읽기 전용) 사용 불가

간 검색 C

1. 벡터 검색이 주요 작업일 때
 - 실시간 검색 API
 - 빠른 응답 시간 필수
 - 1초 이상 대기 불가능
2. 매우 큰 데이터 (수천만 벡터)
 - 메모리 부족 → SSD 스펴
 - 성능 급격히 저하
3. 높은 처리량 필요할 때
 - 초당 수천 쿼리
 - 래치 경합 심각
4. 성능이 사업에 직결될 때
 - 검색 성능 = 사용자 경험
 - 100ms vs 1000ms의 차이가 큼

C

- pgvector는:
- 범용성 + 약간의 벡터 기능 필요
 - 성능보다 SQL 통합이 우선

이 경우에 선택할 것.

- 벡터 성능이 중요하면:
- Faiss/Milvus (전문 DB)
 - pgvector + Exqutor (하이브리드)

p98Ld d T

KCPq C

Pq C

1. 인메모리만 지원
 - 데이터가 메모리에 모두 들어가야 함
 - 1000만 벡터 = ~90GB 메모리 필요

- 서버 메모리가 512GB라도, 대규모는 불가능
2. 읽기 전용 - 데이터 추가/수정 불가능 - 벡터 데이터가 고정되어야 함 - 실시간 업데이트 필요하면 전체 재구축
3. SQL 미지원 - "embedding < 0.5 AND created_at > '2024-01-01'" - 이런 복합 쿼리 불가능 - 애플리케이션에서 별도 필터링 필요
4. ACID 미지원 - 트랜잭션 없음 - 일관성 보장 없음 - 금융 데이터 등에 사용 불가
5. 단일 머신만 지원 - 분산 처리 불가능 - 대규모 시스템에서 확장 어려움

간접적 방법 C

1. 디스크 지원 - 데이터 크기 제약 없음 - SSD/HDD에 저장 가능
2. 동적 업데이트 - 실시간 벡터 추가/수정 가능 - 뮤지컬 임베딩 같이 자주 갱신되는 데이터
3. SQL 통합 - 복합 쿼리 가능 - WHERE/JOIN/GROUP BY 모두 사용
4. ACID 지원 - 트랜잭션 안전성 - 금융/의료 데이터에 안전
5. 분산 지원 (Citus 등) - PostgreSQL을 여러 서버로 확장 가능

C
Faiss vs pgvector: - Faiss는 "벡터 특화, 제한 많음" - pgvector는 "범용성, 약간 느림"
선택: 벡터만 필요하고 메모리 충분: Faiss 벡터 + SQL 통합 필요: pgvector 벡터 성능 + SQL 통합: pgvector + Exqutor

p99Latency는 2년 B T

KC : - . C

: C

방법: ECQO (Estimated Cardinality Query Optimization)
동작: 1. 쿼리 실행 중 벡터 필터의 정확한 선택도 학습 2. 중간 결과의 카디널리티 정확히 파악 3. 이후 조인 순서 동적 재최적화 4. 비효율적 계획 → 효율적 계획으로 변경

- 결과:
- 선택도 추정 오류: 33,300배 → 0배 (정확)
 - 쿼리 성능: 425배 향상

C

- 한계 1 (버퍼 풀 래치): 미해결
- 근본 한계라 Exqutor로 불가능
 - 약 20배 오버헤드 여전
- 한계 2 (MVCC): 미해결
- 근본 한계
 - 약 30% 오버헤드 여전
- 한계 3 (공간 증폭): 미해결
- 근본 한계
 - 약 8배 오버헤드 여전
- 한계 4 (인덱스 구축): 미해결
- 별도 최적화 필요 (Exqutor의 범위 밖)
 - 약 96배 느림 여전

p9: L T

KC C

6 - .C

- 근본 한계: 완전히 제거 불가능
- 해결책:
- VBASE: 벡터 전용 메모리 영역 (일반 버퍼 풀 우회)
 - 아키텍처 수준의 변경 필요
 - PostgreSQL 핵심 개발자 협력 필수

- 기대 효과:
- 20배 오버헤드 → 1배 정도로 감소

7 -WeMMC

- 근본 한계: 완전히 제거 불가능
- 해결책:
- 벡터 데이터를 immutable로 취급
 - MVCC 체크 단순화 (변경 불가능이므로 버전 선택 불필요)
 - 메타데이터 축소 가능
- 기대 효과:
- 30% 오버헤드 → 5% 정도로 감소

8 - .C

- 근본 한계: 8KB 페이지 기반 저장 구조 근본 문제
- 해결책:
- 벡터 전용 저장소 설계
 - 가변 크기 저장 (동시성 관리 복잡도 증가)
 - 또는 페이지 크기 조정 (PostgreSQL 버전별로 다름)

- 기대 효과:
- 8배 오버헤드 → 2배 정도로 감소

9 - .C

부분 해결 가능:

- 방법 1: WAL 우회 (인덱스 구축 시)
- 인덱스는 재생성 가능하므로 로그 불필요
 - 약 50배 개선

방법 2: 병렬 구축

- 여러 프로세스 동시 작업
- 약 4배 개선 (4 프로세스)

방법 3: 배치 처리

- 벡터 일괄 삽입
- 약 10배 개선

총합: 96배 → 약 5배 정도로 감소 가능

0개입력받 C

Exqutor는 "한계 5만" 집중 해결

- 425배 성능 향상 (가장 큼)
- 다른 한계는 별도 해결 필요

하지만 장점:

- 구현 비용 낮음 (쿼리 최적화만)
- 즉시 적용 가능
- 다른 개선과 조합 가능

결과:

pgvector (기본): 1,000배 느림
pgvector + Exqutor: 20배 느림
pgvector + Exqutor + 다른 최적화: 5배 이하도 가능

B6 B 1

3 0개입력받 k m 3

썬 각개입b`V

#-S[NO 75791] 건발건 d가입개입 3

3

L나능강년 T

3] 각개입b`V 간검s개입

Pq개

655

6555

3

실제 성능:

- Faiss (메모리 기반): 100ms
- pgvector (디스크 기반): 1,100초
- 비율: 11,000배

#

3

1

H

3

LB

:

3

3

C

-Vq겨y.

] 각개입b`V

#

#

3

3

-RYbf .

3

4

3

C 75

C

] 각개입b`V

개개개개q게

3WeMM-

4

1

3

C 85)

C

] 각개입b`V

AUL

3RYbf

3

6555

Pq개 B5QL

] 각개입b`V

755QL

3

C A 1 bbN 6555

C

f KV-f 같아2 Kyuqt V각. 3 6555
9A -Pq2개 85 . 3

C B=

C

3] 각머갈b`V - . #883) # 3
5356) 계6) 3

예시: 1000만 벡터에서 가까운 100개 찾기

- pgvector 추정: 333만 행 (33.3%)
- 실제: 100행 (0.001%)
- 오류: 33,300배!

3655 각머갈u갈 =55 -가b가→ 1간검s겨갈 =

- 655 .3

C 97:

L 눈

C

- .C

- .

WeMM- .

-AUL .

] 각머갈b`V 3

C

-f KV .

- 4 .

1

3

c 달는그림

0개한겨갈 3 OM` [-0개→q가t Mq갈가h가렛` 건갈겔 건가→2가가가

3

C

63

73

83

C

C88B55 레 5

C97:

C간검s겨갈 655 레 7계

- 1WeMM . 75 1 - . : 5

3

C

63간검s겨갈

73 1

83

0개입건적말 C
63 - :.
73
83
1간검s적말 # # 3 3 #
10개입건적말 3
3

W6
WeMMC 4
f KVCf 갈2Kyuqt V2k1
Vq적yC 1
bu-us가갈적)
Mq갈가적
RYbf CRzu갈갈yzq-Yq갈qr-+b→q-f 적말-1
OM` [CO개입건적말 1

[6
C
63Pq개 H
Pq개 1] 각개갈b`V
- .
73 H

83간검s적말 H
53 3개
53 3개

0개입건적말 C
63
1WeMM1
73
83
W개`V1[갈s+ H